



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

**Кафедра инструментального и прикладного программного обеспечения
(ИиППО)**

КУРСОВАЯ РАБОТА

по дисциплине: Проектирование и разработка клиент-серверных приложений

по профилю: Разработка программных продуктов и проектирование информационных систем

направления профессиональной подготовки: 09.03.04 «Программная инженерия»

Тема: «Клиент-серверное приложение управления рабочими процессами»

Студент: Данов Арсений Иванович

Группа: ИКБО-10-23

Работа представлена к защите _____ (дата) _____ /Данов А.И. /
(подпись и ф.и.о. студента)

Руководитель: ст. преподаватель Сеницын Анатолий Васильевич

Работа допущена к защите _____ (дата) _____ /Сеницын А.В. /
(подпись и ф.и.о. рук-ля)

Оценка по итогам защиты: _____

_____ / ст. преподаватель Сеницын А.В. /
_____ / _____ /

М. РТУ МИРЭА. 2026



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ЗАДАНИЕ
на выполнение курсовой работы

по дисциплине: Проектирование и разработка клиент-серверных приложений
по профилю: Разработка программных продуктов и проектирование информационных систем
направления профессиональной подготовки: Программная инженерия (09.03.04)

Студент: Данов Арсений Иванович

Группа: ИКБО-10-23

Срок представления к защите: 18.05.2026

Руководитель: ст. преп. Сеницын Анатолий Васильевич

Тема: «Клиент-серверное приложение управления рабочими процессами»

Исходные данные: методология двенадцати факторов, Git, Нормативный документ: инструкция по организации и проведению курсового проектирования СМК МИРЭА 7.5.1/04.И.05-18.

Перечень вопросов, подлежащих разработке, и обязательного графического материала: 1. Провести анализ предметной области для выбранной темы. 2. Выбрать клиент-серверную архитектуру для разрабатываемого приложения и дать её детальное описание с помощью UML. 3. Выбрать программный стек для реализации фуллстек CRUD приложения. 4. Разработать клиентскую и серверную части приложения, реализовать авторизацию и аутентификацию пользователя, обеспечить работу с базой данных, заполнить тестовыми данными, валидировать ролевую модель на некорректные данные. 5. Провести фаззинг-тестирование. 6. Разместить исходный код клиент-серверного приложения в системе контроля версий с наличием Dockerfile и описанием структуры проекта в readme файле. 7. Развернуть клиент-серверное приложение в облаке. 8. Разработать презентацию с графическими материалами.

Руководителем произведён инструктаж по технике безопасности, противопожарной технике и правилам внутреннего распорядка.

Зав. кафедрой ИиППО: [подпись] / Болбаков Р. Г. /, «18» февраля 2026 г.

Задание на КР выдал: [подпись] / Сеницын А. В. /, «18» февраля 2026 г.

Задание на КР получил: [подпись] / Данов А. И. /, «18» февраля 2026 г.

РЕФЕРАТ

Отчёт 48 с., 7 рис., 5 табл., 20 ист.

КЛИЕНТ-СЕРВЕРНОЕ ПРИЛОЖЕНИЕ, УПРАВЛЕНИЕ РАБОЧИМИ ПРОЦЕССАМИ, CRUD, DJANGO, DOCKER, ФАЗЗИНГ-ТЕСТИРОВАНИЕ

Данов А.И., курсовая работа по направлению подготовки «Программная инженерия» по дисциплине «Проектирование и разработка клиент-серверных приложений» на тему «Клиент-серверное приложение управления рабочими процессами»: М. 2026 г., МИРЭА - Российский технологический университет (РТУ МИРЭА), Институт информационных технологий (ИИТ), кафедра инструментального и прикладного программного обеспечения (ИиППО).

Целью работы является проектирование и разработка полнофункционального клиент-серверного приложения управления рабочими процессами. В работе проведён анализ предметной области, выбрана и обоснована клиент-серверная архитектура, описана с помощью UML-диаграмм. Определён программный стек для реализации fullstack CRUD-приложения: Django, PostgreSQL, Redis, Nginx, Traefik, Docker. Разработаны клиентская и серверная части приложения с реализацией аутентификации, ролевой модели, полного набора CRUD-операций для задач и категорий, валидацией данных. Проведено фаззинг-тестирование с использованием библиотеки Hypothesis. Исходный код размещён в системе контроля версий GitHub, приложение контейнеризировано с использованием Docker и развёрнуто на виртуальной машине Yandex Cloud.

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	7
1 ВВЕДЕНИЕ	10
2 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	12
2.1 Характеристика предметной области	12
2.2 Основные участники и бизнес-процессы	12
2.3 Анализ существующих решений	13
2.4 Формирование требований к системе	15
2.4.1 Функциональные требования	15
2.4.2 Нефункциональные требования	15
2.4.3 Матрица соответствия требований	16
2.5 Выводы по главе	16
3 ПРОЕКТИРОВАНИЕ КЛИЕНТ-СЕРВЕРНОЙ АРХИТЕКТУРЫ	18
3.1 Обоснование выбора клиент-серверной архитектуры	18
3.1.1 Трёхуровневая модель архитектуры	18
3.1.2 UML-описание архитектуры	19
3.2 Описание клиентской части	21
3.3 Выбор программного стека	22
3.3.1 Серверный фреймворк	22
3.3.2 СУБД	23
3.3.3 Кэш и вспомогательные сервисы	23
3.3.4 Итоговый стек технологий	24
3.4 Выбор системы контроля версий	24
3.5 Выбор облачной платформы	25
3.6 Выводы по главе	25
4 РЕАЛИЗАЦИЯ FULLSTACK CRUD-ПРИЛОЖЕНИЯ	27
4.1 Структура проекта	27
4.2 Реализация моделей данных	27
4.3 Реализация полного CRUD для задач	28
4.4 Реализация CRUD для категорий задач	29
4.5 Реализация редактирования профиля	30
4.6 Реализация аутентификации и ролевой модели	31

4.6.1 Аутентификация	31
4.6.2 Ролевая модель и разграничение доступа	32
4.6.3 Валидация ролевой модели на некорректные данные	33
4.7 Реализация страниц и интерфейса	33
4.7.1 Страница профиля сотрудника	33
4.7.2 Страница статистики и задач сотрудника	34
4.7.3 Диаграмма последовательности добавления задачи	35
4.8 Реализация статистики и кэширования	36
4.9 Тестовые данные	36
4.10 Выводы по главе	37
5 ТЕСТИРОВАНИЕ И РАЗВЁРТЫВАНИЕ ПРИЛОЖЕНИЯ	38
5.1 Фаззинг-тестирование	38
5.1.1 Подход к тестированию	38
5.1.2 Реализованные тесты	38
5.2 Размещение кода в системе контроля версий	40
5.2.1 Структура репозитория	40
5.2.2 История коммитов	40
5.3 Контейнеризация приложения	41
5.3.1 Dockerfile	41
5.3.2 Docker Compose	41
5.4 Развёртывание на Yandex Cloud	42
5.4.1 Процесс развёртывания	42
5.4.2 Результат развёртывания	43
5.5 Выводы по главе	43
Заключение	45
Список использованных источников	47

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящем отчете применяют следующие определения, обозначения и сокращения.

API (Application Programming Interface)	– программный интерфейс приложения, обеспечивающий взаимодействие между программными компонентами.
CBV (Class-Based Views)	– подход к написанию представлений в Django с использованием классов вместо функций; обеспечивает повторное использование кода через наследование.
CRUD (Create, Read, Update, Delete)	– базовые операции создания, чтения, изменения и удаления данных.
CSS (Cascading Style Sheets)	– язык описания внешнего вида веб-страниц.
Django	– высокоуровневый веб-фреймворк на языке Python, предназначенный для разработки серверной части веб-приложений по архитектурному паттерну MVT.
Docker	– программная платформа контейнеризации, используемая для упаковки и запуска приложений в изолированной среде.
Docker Compose	– инструмент для описания и запуска многоконтейнерных приложений с помощью единого конфигурационного YAML-файла.
Dockerfile	– текстовый файл с инструкциями для сборки Docker-образа приложения.
Fullstack-приложение	– приложение, включающее как клиентскую (frontend), так и серверную (backend) части.
Gunicorn	– WSGI-совместимый сервер для запуска Python-приложений в продуктивной среде.
HTML (HyperText Markup Language)	– язык разметки веб-страниц.

HTTP (HyperText Transfer Protocol)	– протокол передачи данных между клиентом и сервером.
MVT (Model-View-Template)	– архитектурный паттерн, используемый в Django и разделяющий приложение на модели данных, обработчики запросов и шаблоны отображения.
Nginx	– высокопроизводительный веб-сервер и обратный прокси-сервер, используемый для отдачи статических файлов и балансировки нагрузки.
ORM (Object-Relational Mapping)	– технология отображения объектов языка программирования на таблицы реляционной базы данных.
PaaS (Platform as a Service)	– модель облачных вычислений, при которой поставщик предоставляет платформу для разработки, запуска и управления приложениями.
PostgreSQL	– объектно-реляционная система управления базами данных с открытым исходным кодом.
Property-based testing	– подход к тестированию, при котором проверяются свойства программы на автоматически сгенерированных входных данных.
Redis	– высокопроизводительное хранилище данных в памяти, применяемое для кэширования и обмена сообщениями.
SQL (Structured Query Language)	– язык запросов к реляционным базам данных.
Traefik	– современный обратный прокси и балансировщик нагрузки, ориентированный на работу в контейнерных средах.
UML (Unified Modeling Language)	– унифицированный язык моделирования, используемый для визуального описания архитектуры программных систем.
URL (Uniform Resource Locator)	– унифицированный указатель адреса ресурса в сети.

VCS (Version Control System)	– система контроля версий, обеспечивающая отслеживание изменений в исходном коде.
WSGI (Web Server Gateway Interface)	– стандарт взаимодействия между веб-сервером и Python-приложением.
БД	– база данных.
СУБД	– система управления базами данных.
Фаззинг-тестирование (fuzz testing)	– метод тестирования программного обеспечения, при котором на вход подаются случайные, граничные или некорректные данные с целью выявления ошибок обработки.
Фреймворк	– программный каркас, задающий базовую структуру и инструменты разработки программного обеспечения.

1 ВВЕДЕНИЕ

В современных организациях эффективное управление рабочими процессами является одним из ключевых условий производительности команды. Учёт выполненных задач, фиксация затраченного рабочего времени, контроль категорий работ и формирование сводной отчётности — всё это задачи, которые требуют надёжного программного инструмента. Ручной учёт в таблицах и документах не обеспечивает целостности данных, затрудняет контроль сроков и не позволяет получить актуальную управленческую информацию в режиме реального времени [1].

Клиент-серверная архитектура является стандартом разработки подобных веб-приложений: она позволяет централизованно хранить данные на сервере, предоставляя к ним доступ множеству пользователей через браузер. При этом логика обработки данных, разграничение доступа и взаимодействие с базой данных реализуются на стороне сервера, а пользовательский интерфейс — на клиентской стороне [2].

Актуальность темы обусловлена необходимостью реализации полнофункционального fullstack-приложения, охватывающего все уровни клиент-серверной системы: клиентский интерфейс, серверную логику, реляционную базу данных и развёртывание в облачной среде. Дополнительным требованием является обеспечение качества разрабатываемого программного обеспечения путём применения современных методов тестирования, включая фаззинг [3].

Целью курсовой работы является проектирование и разработка клиент-серверного приложения управления рабочими процессами, обеспечивающего полный набор CRUD-операций, ролевую модель доступа и облачное развёртывание.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области управления рабочими процессами;
- выбрать клиент-серверную архитектуру и дать её детальное описание с помощью UML;

- выбрать программный стек для реализации fullstack CRUD-приложения;
- разработать клиентскую и серверную части приложения, реализовать аутентификацию и авторизацию, обеспечить работу с базой данных, заполнить тестовыми данными, валидировать ролевую модель;
- провести фаззинг-тестирование приложения;
- разместить исходный код в системе контроля версий с наличием Dockerfile и описанием структуры проекта в README;
- развернуть приложение в облаке.

Объект исследования — процессы управления рабочими задачами и учётом рабочего времени сотрудников в организации.

Предмет исследования — методы и средства проектирования и разработки клиент-серверного приложения на базе фреймворка Django с полным циклом от проектирования архитектуры до облачного развёртывания.

Результатом работы является полнофункциональное клиент-серверное приложение TaskAndTimes, обеспечивающее регистрацию сотрудников, управление задачами и категориями работ, отображение статистики, ролевое разграничение доступа, фаззинг-тестирование и контейнеризированное развёртывание в облаке.

2 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

2.1 Характеристика предметной области

Управление рабочими процессами в современных организациях охватывает задачи планирования, распределения и учёта выполняемых работ. Особую роль играет контроль трудозатрат: руководителю необходимо знать, какие задачи были выполнены, сколько часов на них затрачено и каков расчётный объём вознаграждения. В условиях распределённых команд и удалённой работы потребность в централизованном хранении и отображении этих данных становится ещё более значимой [1].

Во многих небольших организациях и проектных командах подобный учёт ведётся вручную: с помощью электронных таблиц, мессенджеров или текстовых отчётов. Такой подход не обеспечивает целостности данных, создаёт риск дублирования информации, усложняет контроль доступа и требует дополнительных временных затрат на подготовку сводной отчётности [4]. Кроме того, вручную невозможно эффективно связать сведения о сотруднике, категории выполненной задачи, потраченном времени и расчётной стоимости работ.

Следовательно, возникает необходимость в разработке специализированного клиент-серверного приложения, которое будет централизованно хранить данные о сотрудниках и задачах, обеспечивать регистрацию и аутентификацию пользователей, предоставлять полный набор операций по созданию, чтению, редактированию и удалению данных, а также реализовывать ролевое разграничение доступа между сотрудниками и руководителем.

2.2 Основные участники и бизнес-процессы

В рамках рассматриваемой предметной области выделяются следующие основные участники системы:

- сотрудник;
- руководитель;
- администратор системы.

Сотрудник взаимодействует с системой для ведения собственной трудовой отчётности. После регистрации и аутентификации он получает

доступ к личному профилю, где может просматривать сводные показатели своей активности, добавлять, редактировать и удалять сведения о выполненных задачах. Для каждой задачи фиксируются её категория, количество затраченных часов, поясняющий комментарий и при необходимости прикрепляемый отчётный файл. Кроме того, сотрудник может редактировать данные своего профиля и изменять пароль.

Руководитель использует систему для получения управленческой информации. В отличие от сотрудника, он имеет доступ к сводной статистике по всем сотрудникам: количество выполненных задач, суммарно отработанное время и ориентировочная сумма выплат. Руководителю также доступен переход к детализации задач конкретного сотрудника с возможностью редактирования или удаления любой записи. Помимо этого, руководитель управляет справочником категорий задач: создаёт новые категории, изменяет их названия и стоимость, удаляет устаревшие.

Администратор системы обеспечивает поддержку данных через встроенную административную панель Django, которая предоставляет полный доступ ко всем моделям системы.

Ключевые бизнес-процессы рассматриваемой системы:

- регистрация нового сотрудника в системе;
- аутентификация сотрудника или руководителя;
- добавление, редактирование и удаление записи о выполненной задаче;
- редактирование профиля пользователя и смена пароля;
- просмотр руководителем сводной статистики по сотрудникам;
- управление справочником категорий задач;
- просмотр детализированной информации по задачам конкретного сотрудника.

2.3 Анализ существующих решений

Для определения требований к разрабатываемому приложению рассмотрены существующие программные продукты, применяемые для учёта задач, рабочего времени и управленческой отчётности. В качестве аналогов выбраны следующие решения:

- Jira с модулем учёта трудозатрат Tempo Timesheets [5];

- Bitrix24 [6];
- Clockify [7].

Jira и Tempo Timesheets широко применяются в командах разработки. Их сильной стороной является развитый механизм работы с задачами, проектами и отчётностью. Однако такое решение ориентировано на сложные процессы управления проектами и может оказаться избыточным для небольших коллективов, которым нужен специализированный сервис с понятным набором функций [8].

Bitrix24 представляет собой комплексную корпоративную платформу с функциями коммуникации, CRM и кадрового управления. Преимуществом решения является многофункциональность, однако для локального сценария учёта задач такой продукт перегружен избыточными модулями [6].

Clockify ориентирован на учёт рабочего времени и формирование отчётности. Решение удобно для фиксации времени, но не покрывает задачи управления категориями задач, гибкой настройки ролей и редактирования записей в рамках единого интерфейса [7].

Сравнительный анализ аналогов представлен в таблице 1.

Таблица 1 – Сравнительный анализ аналогов

Решение	Учёт задач (CRUD)	Учёт времени	Сводная статистика	Применимость для специализированного сервиса
Jira + Tempo Timesheets	Высокий	Высокий	Высокий	Средняя
Bitrix24	Высокий	Средний	Высокий	Низкая
Clockify	Низкий	Высокий	Средний	Средняя
Разрабатываемое приложение	Необходимый	Необходимый	Необходимый	Высокая

На основании анализа можно сделать вывод, что существующие решения либо ориентированы на широкий спектр корпоративных задач, либо покрывают только часть необходимой функциональности. Разрабатываемое приложение должно занять нишу специализированного инструмента с полным CRUD-управлением задачами и категориями, ролевой моделью и облачным развёртыванием.

2.4 Формирование требований к системе

На основе анализа предметной области и рассмотренных аналогов сформулированы требования к разрабатываемому приложению.

2.4.1 Функциональные требования

Разрабатываемое приложение должно обеспечивать:

- регистрацию пользователя с вводом имени, фамилии, электронной почты и фотографии;
- аутентификацию сотрудника и руководителя через отдельные точки входа;
- хранение учётных данных и профильной информации пользователя;
- редактирование профиля пользователя и смену пароля;
- создание сотрудником записи о выполненной задаче с указанием категории, затраченного времени, комментария и прикрепляемого файла отчёта;
- редактирование и удаление собственных задач сотрудником;
- просмотр сотрудником собственной статистики и списка задач;
- просмотр руководителем списка сотрудников со сводной информацией по задачам и выплатам;
- редактирование и удаление задач любого сотрудника руководителем;
- управление справочником категорий задач (создание, редактирование, удаление) руководителем;
- расчёт суммарного количества часов и суммы выплат на основании сохранённых записей.

2.4.2 Нефункциональные требования

К основным нефункциональным требованиям относятся:

- доступность веб-сервиса не ниже 99% в рабочее время организации;
- время формирования страницы профиля не более 2 с при объёме данных до 10^4 записей;
- время формирования страницы статистики не более 3 с при количестве сотрудников до 100;
- поддержка не менее 50 одновременно активных пользователей без нарушения целостности данных;

- размер одного прикрепляемого отчётного файла не более 10 МБ;
- хранение данных при перезапуске сервиса за счёт использования постоянного хранилища базы данных;
- защита от несанкционированного доступа: попытка обратиться к чужим данным должна возвращать ошибку 403.

2.4.3 Матрица соответствия требований

Результаты анализа требований представлены в таблице 2.

Таблица 2 – Матрица соответствия требований

Требование	Сотрудник	Руководитель	Серверная часть
Регистрация и аутентификация	Да	Да	Да
Редактирование профиля	Да	Нет	Да
Добавление задачи	Да	Нет	Да
Редактирование собственных задач	Да	Нет	Да
Редактирование задач любого сотрудника	Нет	Да	Да
Удаление задач	Да (своих)	Да (любых)	Да
Просмотр личной статистики	Да	Нет	Да
Просмотр общей статистики	Нет	Да	Да
Управление категориями	Нет	Да	Да
Расчёт суммарных выплат	Нет	Да	Да
Хранение отчётных файлов	Да	Да	Да

Таким образом, серверная часть приложения несёт основную нагрузку по реализации бизнес-логики, хранению данных и разграничению доступа, при этом клиентская часть обеспечивает удобный интерфейс для всех ролей пользователей.

2.5 Выводы по главе

В результате анализа предметной области установлено, что для управления рабочими процессами организации требуется специализированное клиент-серверное приложение с централизованным хранением данных, полным набором CRUD-операций и ролевым разграничением доступа. Рассмотрение существующих решений показало, что универсальные корпоративные платформы обладают избыточной функциональностью, а специализированные сервисы не всегда позволяют адаптировать систему под конкретные требования.

На основе проведённого анализа выделены основные участники системы, определены ключевые бизнес-процессы, а также сформированы функциональные и нефункциональные требования. Полученные результаты служат основанием для выбора архитектуры, технологий реализации и проектирования структуры приложения.

3 ПРОЕКТИРОВАНИЕ КЛИЕНТ-СЕРВЕРНОЙ АРХИТЕКТУРЫ

3.1 Обоснование выбора клиент-серверной архитектуры

Разрабатываемое приложение управления рабочими процессами по своей природе является многопользовательской системой: в ней одновременно работают сотрудники и руководители, данные должны быть централизованы и доступны с любого устройства, а логика разграничения доступа должна быть надёжно защищена на стороне сервера. Именно эти требования обусловили выбор клиент-серверной архитектуры.

Клиент-серверная архитектура предполагает явное разделение системы на две части: клиент, который инициирует запросы и отображает результат, и сервер, который обрабатывает запросы, выполняет бизнес-логику и управляет данными [9]. В рамках данного приложения роль клиента выполняет веб-браузер пользователя, а роль сервера — набор сервисов, развёрнутых в контейнерах.

По сравнению с альтернативами клиент-серверная архитектура имеет ряд ключевых преимуществ для данной задачи:

- централизованное хранение данных исключает дублирование и рассинхронизацию сведений о задачах и сотрудниках;
- разграничение доступа реализуется на стороне сервера, что исключает возможность его обхода на клиенте;
- обновление бизнес-логики и данных происходит в одном месте и немедленно становится доступным всем пользователям;
- клиент не требует установки — достаточно браузера, что снижает порог входа для пользователей.

3.1.1 Трёхуровневая модель архитектуры

Разрабатываемое приложение построено по трёхуровневой (three-tier) клиент-серверной модели, которая разделяет систему на три самостоятельных уровня [9]:

Уровень представления (Presentation Tier) — клиентская часть, с которой взаимодействует пользователь. В данном проекте клиентом является стандартный веб-браузер. Он отправляет HTTP-запросы и отображает HTML-

страницы, сформированные сервером. Никакой бизнес-логики на клиенте нет: весь рендеринг и вся обработка данных выполняются на стороне сервера.

Уровень логики приложения (Application Tier) — серверная часть, реализованная на Django. На этом уровне выполняется аутентификация пользователей, проверка прав доступа, обработка данных форм, выполнение CRUD-операций через ORM, агрегирование статистики и генерация HTML-ответов. Gunicorn обеспечивает WSGI-интерфейс между веб-сервером и Django. Nginx принимает внешние запросы, отдаёт статические файлы напрямую и проксирует динамические запросы к серверу приложений. Traefik выступает точкой входа и балансировщиком нагрузки.

Уровень данных (Data Tier) — серверы данных. PostgreSQL хранит все данные системы: пользователей, категории задач, задачи. Redis обеспечивает кэширование сформированных страниц для снижения нагрузки на сервер приложений.

3.1.2 UML-описание архитектуры

Общая схема взаимодействия компонентов в клиент-серверной архитектуре приложения представлена на рисунке 1. Диаграмма отражает маршрут HTTP-запроса от браузера пользователя через балансировщик нагрузки, веб-сервер и сервер приложений до базы данных и обратно.

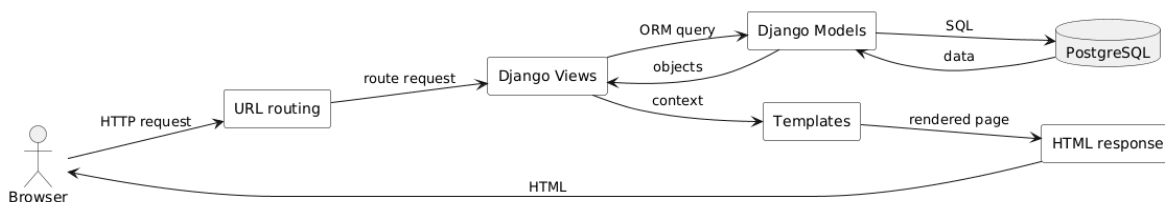


Рисунок 1 – Структурная схема клиент-серверной архитектуры приложения

Листинг 1 – Компонентная диаграмма приложения TaskAndTimes

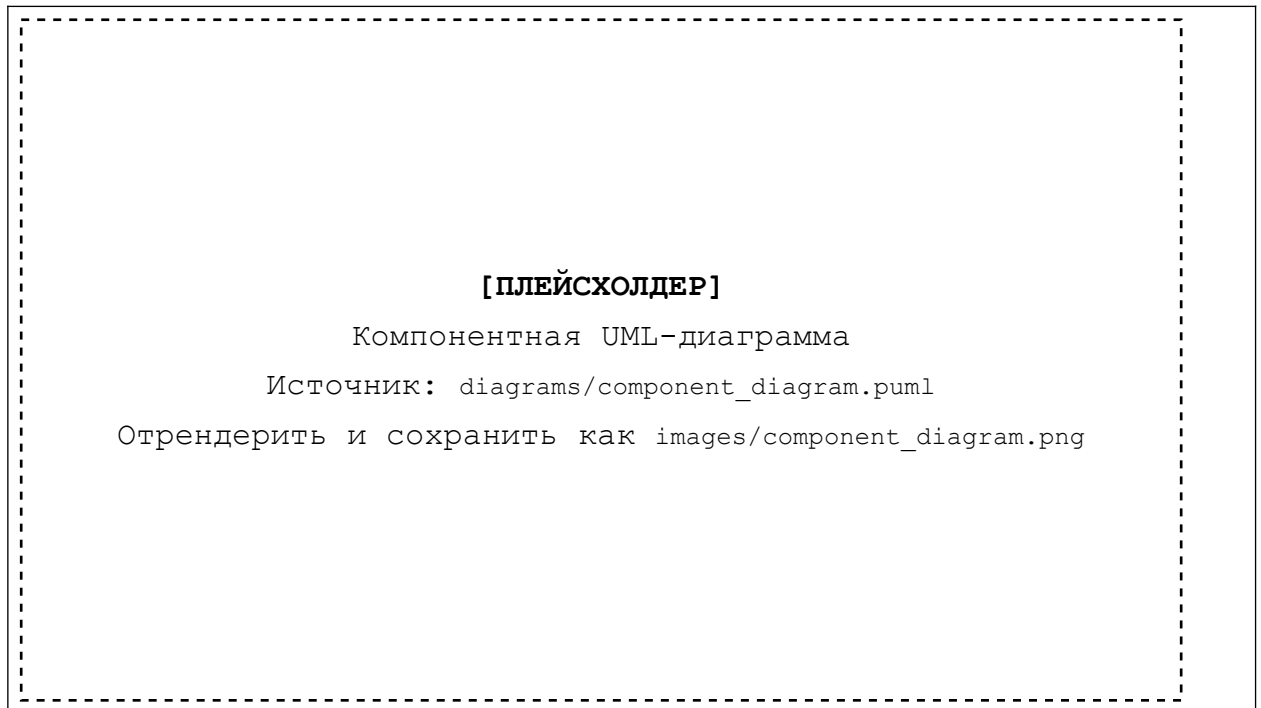


Диаграмма вариантов использования системы представлена на рисунке 2. Она показывает действия, доступные каждому актору: сотруднику, руководителю и администратору.

Листинг 2 – Диаграмма вариантов использования системы



3.2 Описание клиентской части

Клиентская часть приложения реализована на основе серверного рендеринга (Server-Side Rendering): сервер формирует готовые HTML-страницы и передаёт их в браузер пользователя. Такой подход упрощает разработку и обеспечивает корректную работу в любом современном браузере без необходимости установки дополнительного программного обеспечения на стороне клиента [1].

Клиентская часть построена с использованием следующих технологий:

- **HTML** — разметка веб-страниц. Шаблоны Django (*.html) содержат структуру страниц и подставляют данные, переданные из представлений через контекст.
- **CSS** — стилизация интерфейса. Единый файл `static/Tasks/css/styles.css` задаёт оформление всех компонентов: форм, карточек задач, профиля, таблиц статистики.

– **Шаблонный движок Django** — обеспечивает наследование шаблонов (`base.html`), пользовательские теги (`post_form`), условные конструкции и циклы в шаблонах.

Клиентская часть включает следующие основные страницы:

- главная страница с навигацией по точкам входа;
- страница авторизации сотрудника и руководителя;
- страница регистрации;
- страница профиля сотрудника (статистика + форма добавления задачи + список задач);
- страница редактирования профиля и смены пароля;
- страница статистики по всем сотрудникам (для руководителя);
- страница задач конкретного сотрудника (для руководителя);
- страницы редактирования и удаления задачи;
- страницы управления категориями (список, создание, редактирование, удаление);
- страницы ошибок 403 и 404.

Взаимодействие браузера с сервером осуществляется через стандартные HTML-формы: GET-запрос для загрузки страницы, POST-запрос для отправки данных формы. CSRF-защита реализована встроенными средствами Django.

3.3 Выбор программного стека

3.3.1 Серверный фреймворк

Для разработки серверной части рассмотрены Python-фреймворки Django, Flask и FastAPI.

Flask представляет собой легковесный фреймворк с высокой степенью свободы. Однако при разработке приложения с регистрацией, ролевым доступом, административным интерфейсом и ORM разработчик вынужден самостоятельно подбирать и интегрировать большое число дополнительных библиотек [10].

FastAPI ориентирован на разработку API с высокой производительностью и встроенной валидацией данных. Для данного проекта, в котором требуется серверный рендеринг HTML-страниц, форм и шаблонов, FastAPI не является наиболее естественным выбором [11].

Django — высокоуровневый фреймворк, уже содержащий встроенные механизмы для всех ключевых задач: ORM, маршрутизацию, формы, аутентификацию, административную панель и систему шаблонов [2]. В рамках Django применяется паттерн MVT (Model–View–Template), обеспечивающий логичное разделение приложения на данные, обработку запросов и отображение.

Сравнение фреймворков представлено в таблице 3.

Таблица 3 – Сравнение серверных фреймворков

Критерий	Django	Flask	FastAPI
Встроенная ORM	Да	Нет	Нет
Шаблоны и формы	Да	Частично	Нет
Встроенная аутентификация	Да	Нет	Нет
Административная панель	Да	Нет	Нет
Паттерн MVT/MVC	MVT	Нет	Нет
Применимость для проекта	Высокая	Средняя	Низкая

По результатам сравнения для разработки выбран Django.

3.3.2 СУБД

Разрабатываемое приложение хранит структурированные данные о пользователях, категориях и задачах с отношениями между сущностями. Рассмотрены PostgreSQL и MySQL.

PostgreSQL обеспечивает строгий контроль целостности данных, полную поддержку транзакций и эффективное выполнение агрегатных запросов, необходимых для расчёта статистики [12]. Кроме того, PostgreSQL является рекомендованной СУБД для Django в продуктивных средах [13].

MySQL также широко применяется в веб-разработке, однако PostgreSQL предпочтительнее для задач, требующих сложных аналитических запросов и высокой надёжности. По совокупности критериев для проекта выбрана PostgreSQL.

3.3.3 Кэш и вспомогательные сервисы

Redis выбран в качестве кэш-хранилища. Он хранит данные в оперативной памяти, обеспечивает высокую скорость доступа и хорошо

интегрируется с Django через встроенный кэш-бэкенд [14]. Кэширование применяется для страниц профиля, статистики и списка задач.

Nginx выбран в качестве веб-сервера для отдачи статических и медиафайлов, а также обратного прокси для динамических запросов [15]. Traefik используется как балансировщик нагрузки и входная точка, удобно работающая в контейнерной среде Docker [16].

Docker с Docker Compose обеспечивает воспроизводимую сборку и запуск всех сервисов приложения в изолированных контейнерах [17].

3.3.4 Итоговый стек технологий

Итоговый выбор технологий представлен в таблице 4.

Таблица 4 – Итоговый стек технологий

Компонент	Технология	Обоснование выбора
Backend-фреймворк	Django 5.1	Встроенный ORM, аутентификация, шаблоны, MVT
СУБД	PostgreSQL 17	Целостность данных, агрегатные запросы
Кэш	Redis	Быстрый доступ, интеграция с Django
Веб-сервер	Nginx 1.27	Отдача статики, обратный прокси
Балансировщик	Traefik v3.2	Маршрутизация, Docker-интеграция
WSGI-сервер	Gunicorn	Продуктивный запуск Django
Контейнеризация	Docker + Compose	Воспроизводимое развёртывание
Язык программирования	Python 3.12	Поддержка Django, экосистема
Клиентская часть	HTML + CSS + Django Templates	Серверный рендеринг

3.4 Выбор системы контроля версий

Для хранения исходного кода и отслеживания истории изменений выбрана система контроля версий Git как наиболее распространённый инструмент в современной разработке программного обеспечения [18]. В качестве удалённого репозитория используется платформа GitHub.

Применение Git и GitHub обеспечивает следующие возможности:

- полная история изменений кода с содержательными сообщениями коммитов;
- возможность ветвления и слияния для организации работы;

- открытый доступ к исходному коду для демонстрации преподавателю;
- интеграция с системами CI/CD для автоматизированного тестирования и развёртывания.

Исходный код проекта размещён в репозитории по адресу: <https://github.com/Arsenoks4132/TaskMetrics>.

3.5 Выбор облачной платформы

Для развёртывания приложения в продуктивной среде выбрана платформа Yandex Cloud — отечественный облачный провайдер с развитой инфраструктурой. В качестве модели развёртывания выбрана виртуальная машина (Compute Cloud), что обеспечивает полный контроль над окружением и позволяет использовать Docker Compose напрямую без ограничений PaaS-платформ.

Обоснование выбора Yandex Cloud:

- доступность из российских сетей без ограничений;
- поддержка Ubuntu-образов с возможностью установки Docker;
- оплата по факту использования ресурсов;
- простота управления через веб-консоль.

Приложение развёртывается на ВМ с запуском `docker compose up -d --build`, что обеспечивает одновременный старт всех пяти сервисов: Django (×3), Nginx (×3), PostgreSQL, Redis и Traefik. URL задеплоенного приложения будет приведён после завершения развёртывания.

3.6 Выводы по главе

В данной главе обоснован выбор клиент-серверной архитектуры для приложения управления рабочими процессами. Спроектирована трёхуровневая модель системы: уровень представления (браузер), уровень логики приложения (Django + Gunicorn + Nginx + Traefik) и уровень данных (PostgreSQL + Redis). Архитектура описана с помощью UML-диаграмм: структурной схемы взаимодействия компонентов, компонентной диаграммы и диаграммы вариантов использования.

Определён программный стек для реализации fullstack CRUD-приложения, обоснован выбор каждой из технологий. В качестве системы

контроля версий выбран Git с хостингом на GitHub; для облачного развёртывания — виртуальная машина Yandex Cloud.

4 РЕАЛИЗАЦИЯ FULLSTACK CRUD-ПРИЛОЖЕНИЯ

4.1 Структура проекта

Серверная часть приложения реализована на фреймворке Django в виде проекта `TaskAndTime` с единственным приложением `Tasks`. Структура проекта соответствует паттерну MVT: модели описывают данные, представления обрабатывают запросы, шаблоны формируют HTML-ответы.

Все модели предметной области расположены в `Tasks/models.py`, формы — в `Tasks/forms.py`, представления — в `Tasks/views.py`, маршруты — в `Tasks/urls.py`. HTML-шаблоны размещены в `Tasks/templates/Tasks/`, статические файлы — в `static/Tasks/`. Конфигурация проекта сосредоточена в `TaskAndTime/settings.py`.

4.2 Реализация моделей данных

Основу логики хранения данных составляют три модели в файле `Tasks/models.py`.

Модель `User` наследует стандартный класс `AbstractUser`. Это позволяет использовать встроенные механизмы аутентификации Django и одновременно расширить структуру пользователя дополнительным полем `photo` для хранения изображения профиля. Разграничение ролей реализовано через стандартный флаг `is_staff`: сотрудники имеют `is_staff = False`, руководители — `is_staff = True` [19].

Модель `Category` хранит название категории задачи (уникальное, до 100 символов) и стоимость часа работы данной категории в рублях. Наличие этой сущности позволяет переиспользовать категории в нескольких задачах и вычислять стоимость выполненных работ на уровне базы данных.

Модель `Task` хранит информацию о выполненной работе: исполнитель (внешний ключ на `User`), категория (внешний ключ на `Category`), затраченное время в часах, дата создания, текстовый комментарий и прикрепляемый файл отчёта. Для поля `spent` применяются валидаторы `MinValueValidator(1)` и `MaxValueValidator(24)`, ограничивающие допустимое значение интервалом от 1 до 24 часов.

Структура сущностей и связей между ними представлена на ER-диаграмме на рисунке 2.

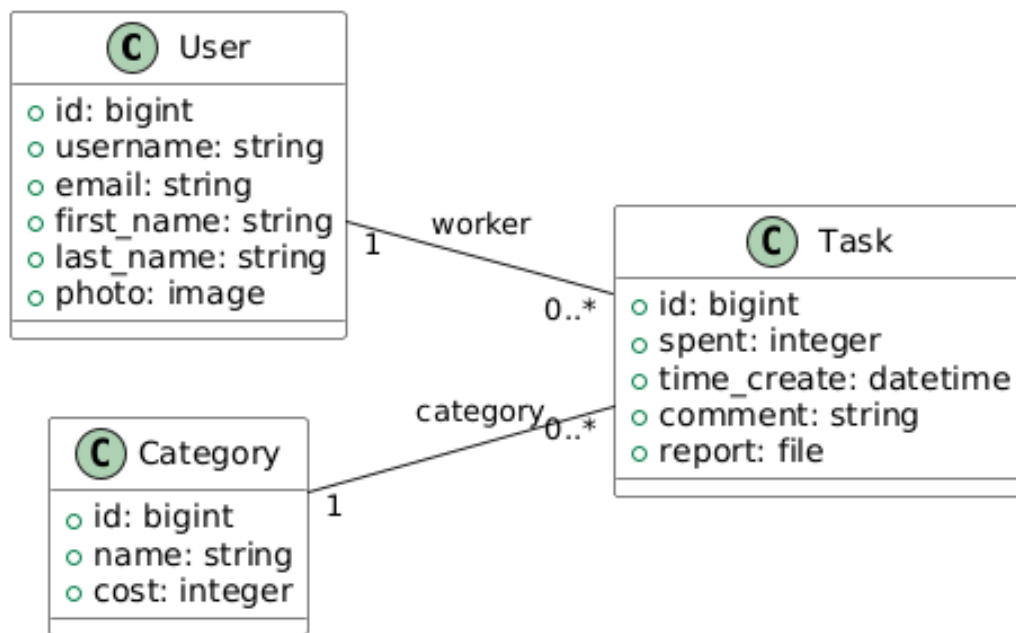


Рисунок 2 – ER-диаграмма приложения TaskAndTimes

4.3 Реализация полного CRUD для задач

Реализация операций Create, Read, Update и Delete для задач является центральной функцией приложения. Все операции защищены проверкой прав доступа.

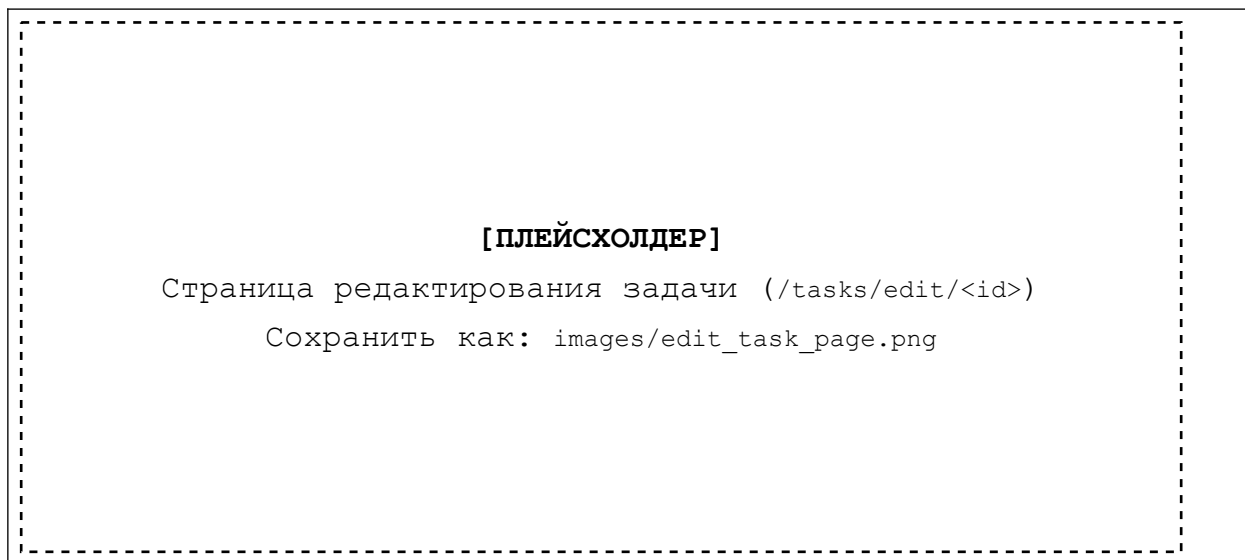
Create — добавление задачи реализовано в представлении `ProfileUser` (класс `CreateView`). При POST-запросе к `/profile` форма `AddTaskForm` проходит валидацию, и если данные корректны, создаётся новый объект `Task` с привязкой к текущему пользователю.

Read — чтение реализовано на нескольких страницах: сотруднику доступен список собственных задач на странице профиля, руководителю — детализация задач любого сотрудника через `TasksList` по URL `/tasks/<employee_id>`.

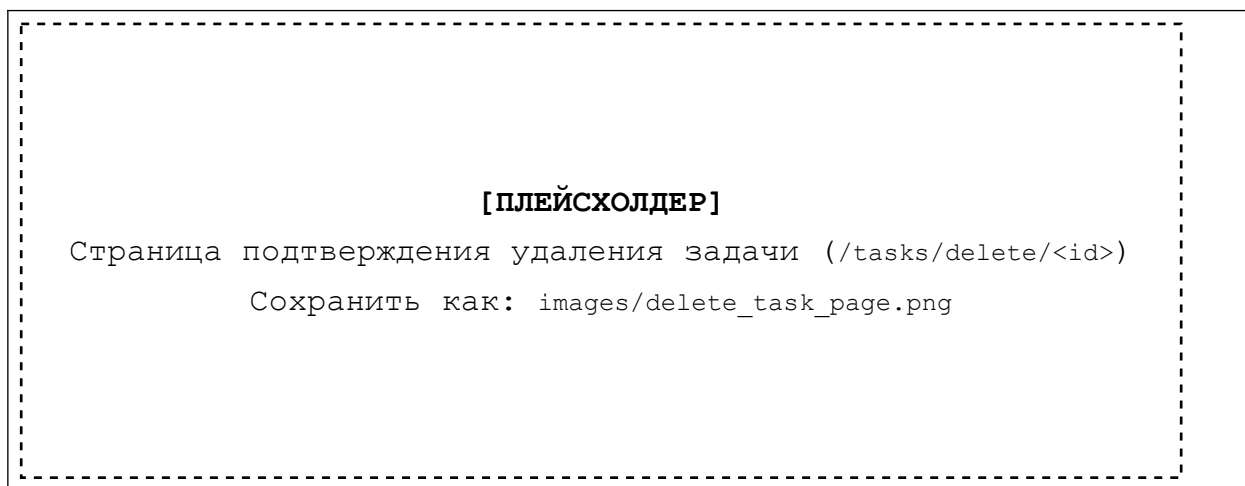
Update — редактирование задачи реализовано в представлении `EditTask` (`UpdateView`), доступном по URL `/tasks/edit/<task_id>`. Представление содержит проверку: если пользователь не является руководителем (`is_staff = False`) и не является владельцем задачи, выбрасывается `PermissionDenied`. Это обеспечивает защиту от несанкционированного редактирования чужих записей. Страница редактирования задачи представлена на рисунке 3.

Delete — удаление реализовано в представлении `DeleteTask` (`DeleteView`), доступном по URL `/tasks/delete/<task_id>`. Применяется та же логика проверки доступа, что и при редактировании. Перед удалением отображается страница подтверждения. Страница подтверждения удаления задачи представлена на рисунке 4.

Листинг 3 – Страница редактирования задачи



Листинг 4 – Страница подтверждения удаления задачи



4.4 Реализация CRUD для категорий задач

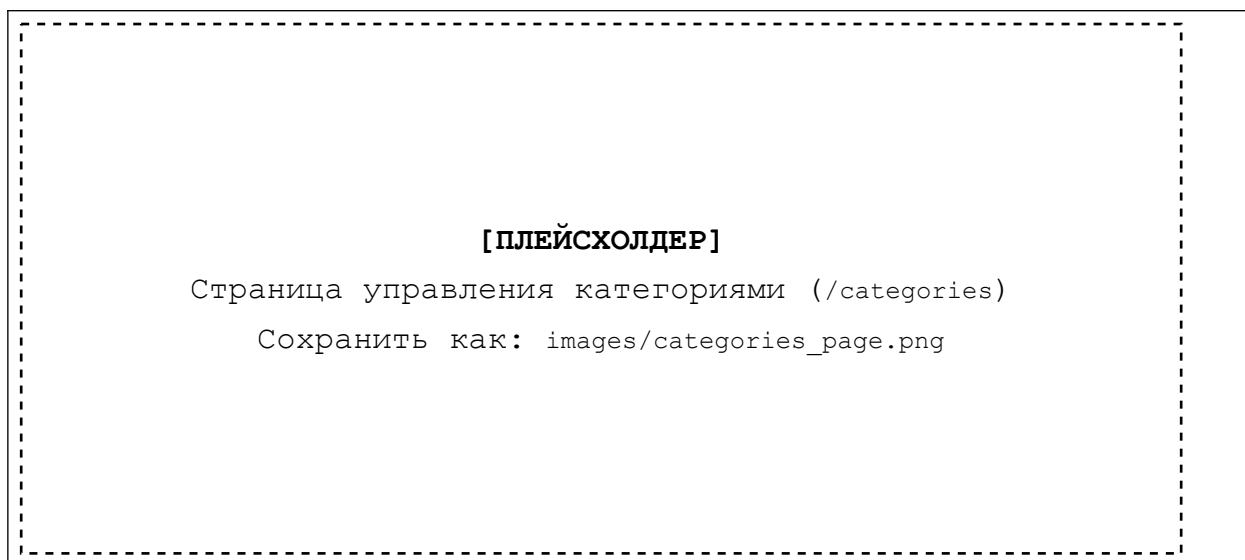
Управление справочником категорий доступно только руководителям. Реализованы четыре операции:

- `CategoryList (/categories)` — список всех категорий с кнопками редактирования и удаления;
- `CreateCategory (/categories/new)` — форма создания новой категории;

– `EditCategory (/categories/edit/<category_id>)` — форма редактирования существующей категории;

– `DeleteCategory (/categories/delete/<category_id>)` — страница подтверждения удаления.

Доступ к каждому из этих представлений защищён миксином `PermissionRequiredMixin` с требованием прав `Tasks.view_user`, которыми обладают только пользователи с флагом `is_staff`. Форма `CategoryForm` выполняет валидацию полей `name` (до 100 символов, уникальность) и `cost` (целое число). Страница управления категориями представлена на рисунке 5. Листинг 5 – Страница управления категориями задач



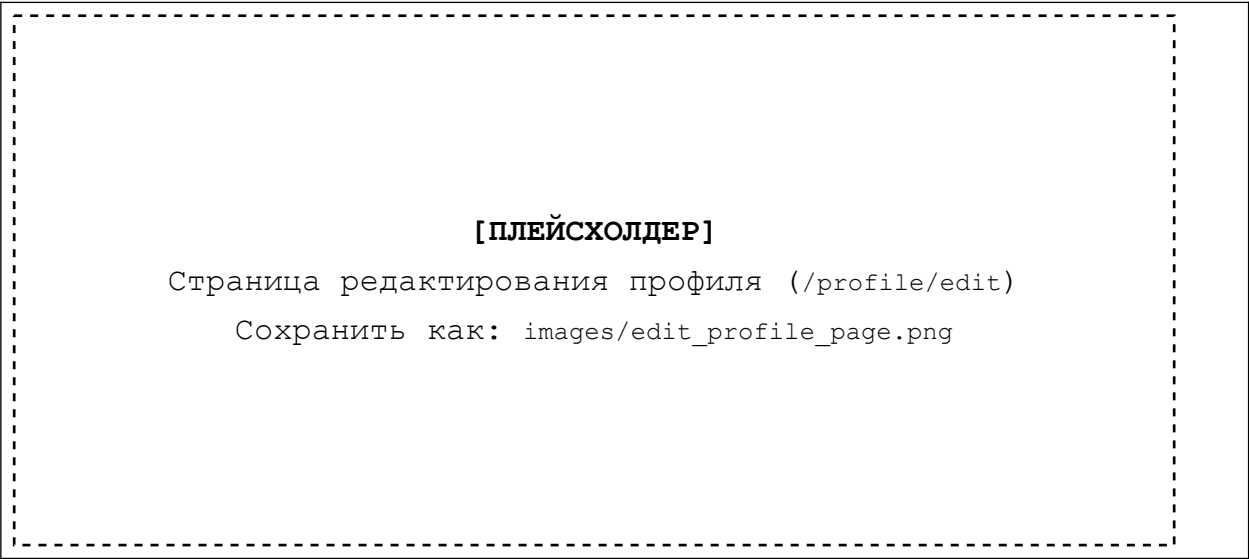
4.5 Реализация редактирования профиля

Сотрудник может редактировать данные своего профиля и изменять пароль. Для этого реализованы два представления.

`EditProfile (/profile/edit)` — основан на `UpdateView` и использует форму `EditProfileForm` с полями `first_name`, `last_name`, `email` и `photo`. Форма содержит валидацию уникальности электронной почты: при попытке установить уже используемый в системе адрес, исключая текущего пользователя, возвращается ошибка валидации.

`ChangePassword (/profile/password)` — основан на `PasswordChangeView`. Сохраняет активность текущей сессии после смены пароля с помощью `update_session_auth_hash`. Страница редактирования профиля представлена на рисунке 6.

Листинг 6 – Страница редактирования профиля пользователя



4.6 Реализация аутентификации и ролевой модели

4.6.1 Аутентификация

Аутентификация реализована с использованием стандартных средств Django: сессионный механизм на основе `django.contrib.auth`. В приложении существуют две точки входа: `/login/employee` (перенаправляет на профиль после успешного входа) и `/login/supervisor` (перенаправляет на страницу статистики).

На рисунке 3 показана страница входа для сотрудника.

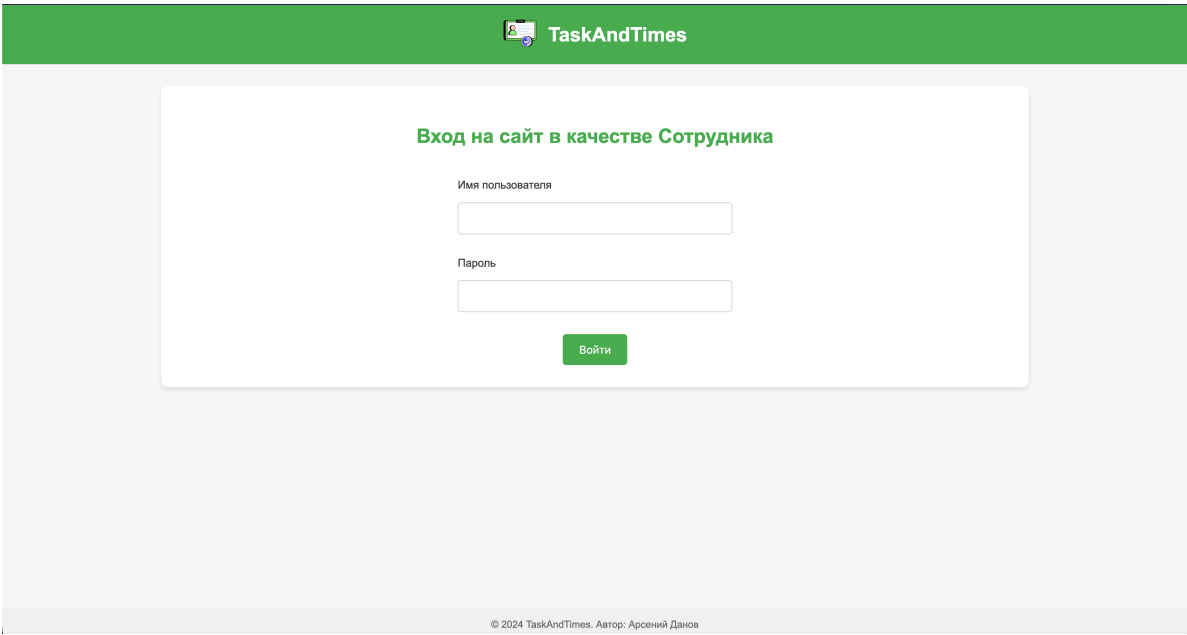
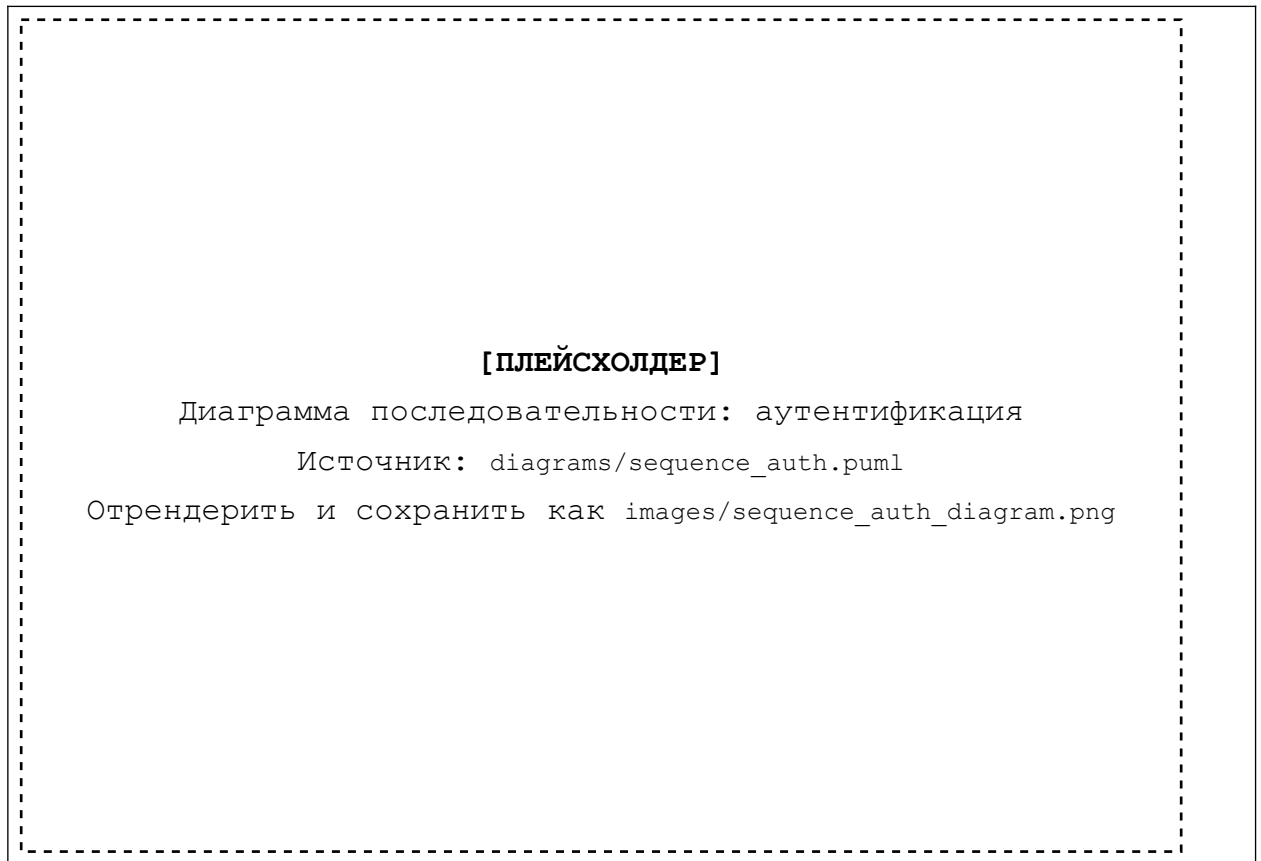


Рисунок 3 – Страница авторизации сотрудника

Диаграмма последовательности процесса аутентификации представлена на рисунке 7.

Листинг 7 – Диаграмма последовательности процесса аутентификации



4.6.2 Ролевая модель и разграничение доступа

В системе выделены три роли: сотрудник (`is_staff = False`), руководитель (`is_staff = True`) и администратор (`is_staff = True, is_superuser = True`). Разграничение доступа реализовано на нескольких уровнях:

- `LoginRequiredMixin` — применяется к странице профиля; не допускает анонимных пользователей.
- `PermissionRequiredMixin(permission_required='Tasks.view_user')` — применяется к страницам статистики, списка задач и управления категориями; доступны только пользователям с `is_staff = True`.
- Явная проверка в `EditTask` и `DeleteTask`: если `not request.user.is_staff and task.worker != request.user`, выбрасывается `PermissionDenied` (HTTP 403) — это исключает редактирование чужих задач сотрудником.

4.6.3 Валидация ролевой модели на некорректные данные

Для обеспечения безопасности системы реализован ряд дополнительных проверок:

- **Валидаторы паролей:** включены все четыре стандартных валидатора Django — проверка схожести с именем пользователя, минимальная длина (8 символов), проверка по списку распространённых паролей, запрет числовых паролей.
- **Уникальность email:** при регистрации и редактировании профиля производится проверка, что указанный адрес электронной почты не занят другим пользователем.
- **Диапазон поля `spent`:** на уровне модели применяются `MinValueValidator(1)` и `MaxValueValidator(24)`, дублирующие граничные ограничения в атрибутах формы (`min=1, max=24`).
- **Длина поля `comment`:** ограничена 1000 символами на уровне модели и формы.

- **Защита от CSRF:** все формы используют Django CSRF-токен.

4.7 Реализация страниц и интерфейса

4.7.1 Страница профиля сотрудника

После аутентификации сотрудник попадает на страницу профиля, где отображаются его персональные данные, сводная статистика (количество задач и суммарные часы), форма добавления новой задачи и список уже добавленных задач с кнопками редактирования и удаления. Страница профиля представлена на рисунке 4.

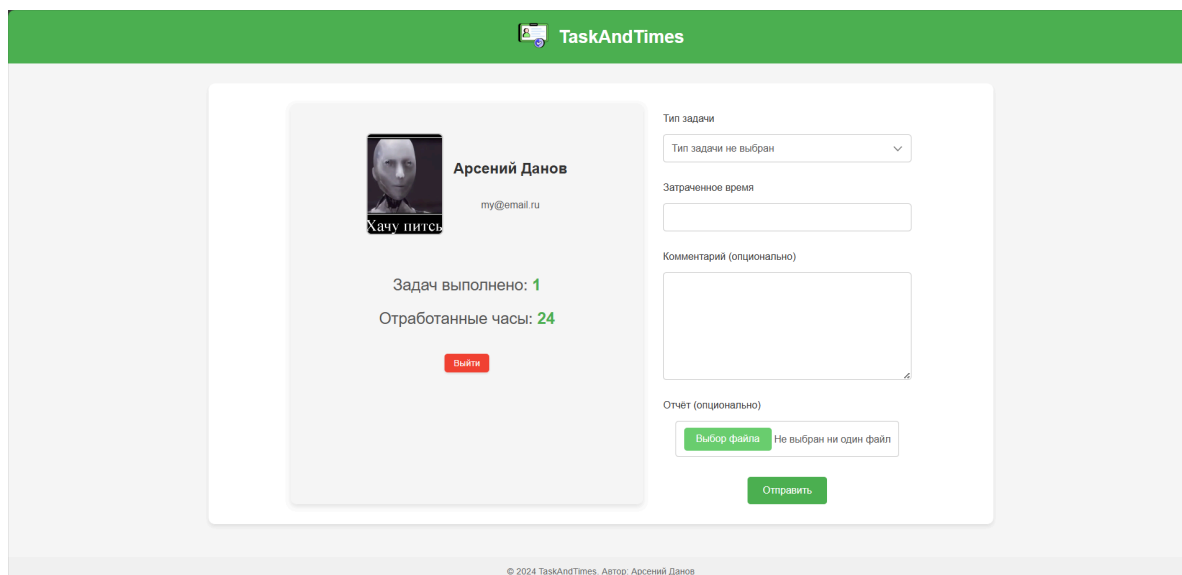


Рисунок 4 – Страница профиля сотрудника

4.7.2 Страница статистики и задач сотрудника

Руководитель видит список всех сотрудников с агрегированными данными: количество задач, суммарные часы и расчётная сумма выплат. При нажатии на карточку сотрудника открывается страница с детализацией задач и кнопками редактирования и удаления. Страницы статистики и задач сотрудника представлены на рисунках 5 и 6.

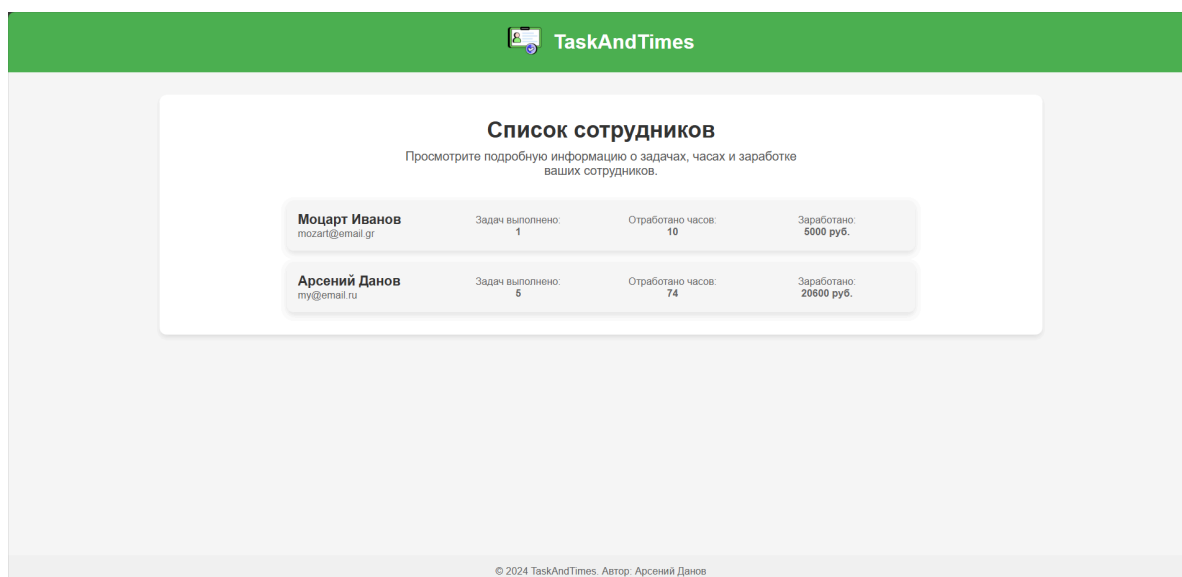


Рисунок 5 – Страница статистики по сотрудникам

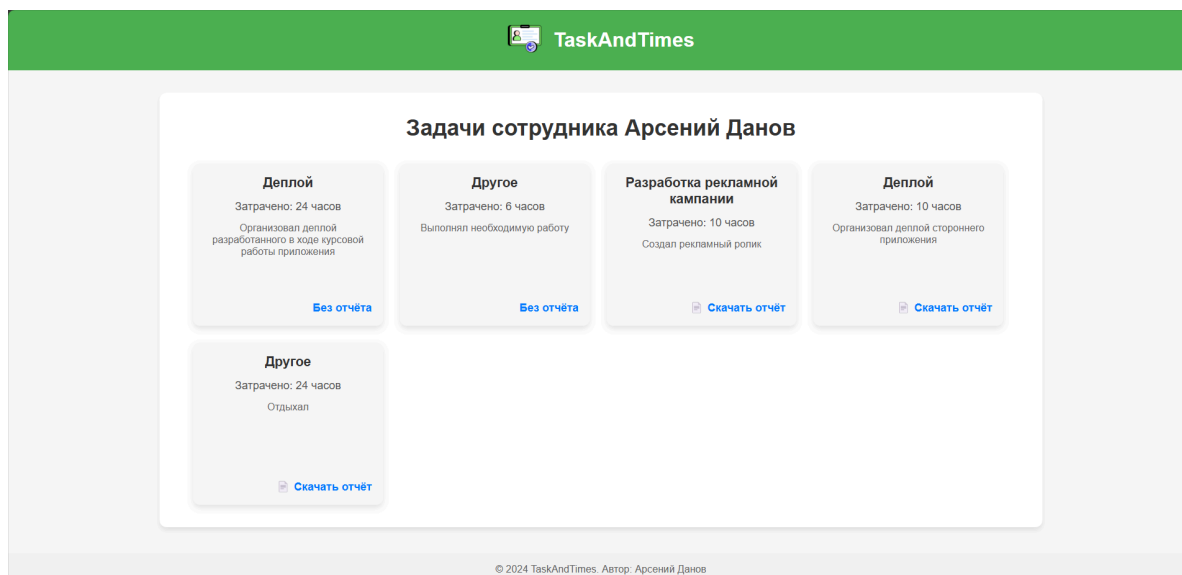


Рисунок 6 – Страница задач конкретного сотрудника

4.7.3 Диаграмма последовательности добавления задачи

Процесс добавления задачи сотрудником описан диаграммой последовательности на рисунке 8.

Листинг 8 – Диаграмма последовательности процесса добавления задачи



4.8 Реализация статистики и кэширования

Одной из ключевых функций приложения является предоставление руководителю агрегированной статистики. Логика реализована в представлении `Statistics` с использованием ORM Django:

- для каждого сотрудника вычисляются суммарное количество задач (`Count`), суммарные часы (`Sum('spent')`) и суммарная расчётная стоимость (`Sum(F('spent') * F('category__cost'))`);
- вычисления выполняются на уровне базы данных PostgreSQL, что снижает объём данных, передаваемых в Python-код.

Для снижения нагрузки на сервер применяется кэширование страниц через Redis. В конфигурации маршрутов декоратор `cache_page` используется для страниц профиля (10 с), статистики (30 с) и списка задач сотрудника (20 с). При изменении данных (добавление, редактирование или удаление задачи) кэш соответствующей страницы инвалидируется.

4.9 Тестовые данные

Для проверки работоспособности приложения используется фикстура `initial_fixture.json`, которая загружается автоматически при первом запуске контейнера (скрипт `entrypoint.sh`). Фикстура содержит:

- несколько категорий задач с различной стоимостью часа (например, «Разработка интерфейсов» — 800 руб./ч, «Тестирование» — 600 руб./ч);
- демонстрационного сотрудника (`employee_demo`);
- демонстрационного руководителя (`manager_demo, is_staff = True`);
- набор задач, привязанных к демонстрационному сотруднику.

Административная панель Django обеспечивает дополнительный способ управления данными системы. Её интерфейс представлен на рисунке 7.

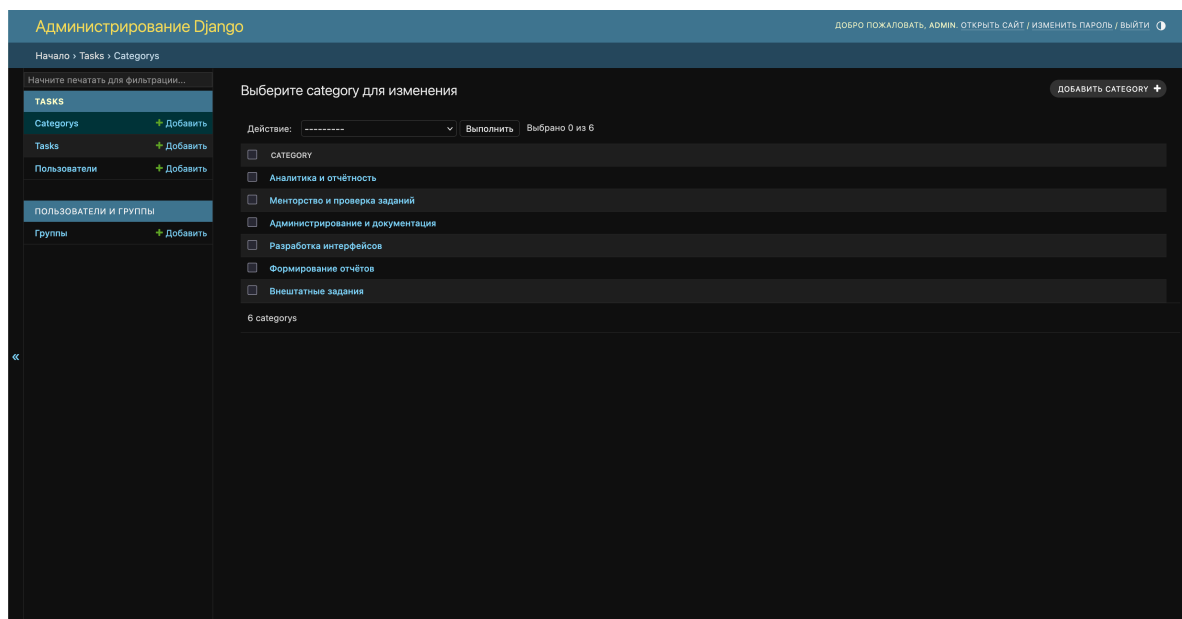


Рисунок 7 – Административная панель Django

4.10 Выводы по главе

В рамках разработки реализовано полнофункциональное fullstack CRUD-приложение на базе Django. Для задач реализованы операции Create, Read, Update и Delete с ролевой защитой доступа: сотрудник управляет только собственными задачами, руководитель — задачами любого сотрудника. Для категорий задач реализован полный CRUD, доступный только руководителям. Добавлено редактирование профиля и смена пароля.

Ролевая модель защищена на нескольких уровнях: через миксины Django, явные проверки `PermissionDenied` и валидацию входных данных (пароли, email, числовые ограничения). Реализованы агрегатные вычисления статистики на уровне базы данных и кэширование страниц через Redis.

5 ТЕСТИРОВАНИЕ И РАЗВЁРТЫВАНИЕ ПРИЛОЖЕНИЯ

5.1 Фаззинг-тестирование

5.1.1 Подход к тестированию

Для обеспечения качества разрабатываемого приложения применяется фаззинг-тестирование (fuzz testing) — метод, при котором на вход программы автоматически подаются случайные, граничные или некорректные данные с целью выявления ошибок обработки, нарушений валидации и некорректного поведения [20].

В качестве инструмента фаззинг-тестирования выбрана библиотека Hypothesis — реализация property-based testing для Python. В отличие от классических unit-тестов с фиксированными входными данными, Hypothesis автоматически генерирует большое количество вариантов входных данных в соответствии с заданными стратегиями и ищет значения, при которых тест не проходит. Найденные контрпримеры минимизируются и сохраняются для воспроизведения.

5.1.2 Реализованные тесты

Тесты размещены в файле `Tasks/tests.py` и разделены на следующие классы.

TaskModelValidationTest — фаззинг модели `Task`, поле `spent`:

- `test_valid_spent_values` — любое целое значение в диапазоне `[1, 24]` должно проходить `full_clean()` без исключения;
- `test_invalid_spent_values` — любое значение вне диапазона `[1, 24]` должно вызывать `ValidationError`;
- `test_task_total_cost_calculation` — произведение `spent × cost` всегда является корректным неотрицательным целым числом.

AddTaskFormFuzzTest — фаззинг формы `AddTaskForm`:

- `test_form_accepts_any_valid_comment` — форма должна быть валидна для любого комментария длиной до 1000 символов;
- `test_form_rejects_invalid_spent` — форма должна возвращать ошибку поля `spent` при значениях вне допустимого диапазона;
- `test_form_rejects_too_long_comment` — форма должна отклонять комментарии длиной свыше 1000 символов.

CategoryFormFuzzTest — фаззинг формы CategoryForm:

- test_valid_category_name — форма валидна для любого непустого названия длиной до 100 символов;
- test_valid_category_cost — форма принимает любое неотрицательное целое значение стоимости;
- test_category_name_too_long — форма отклоняет названия длиннее 100 символов.

RegisterFormEmailFuzzTest — фаззинг поля email формы регистрации:

- test_unique_email_accepted — форма регистрации принимает корректный уникальный адрес электронной почты;
- test_invalid_email_rejected — строки без символа @ должны отклоняться как некорректный email.

RoleBasedAccessTest — проверка ролевой модели на несанкционированный доступ:

- test_anonymous_profile_redirects_to_login — неаутентифицированный запрос к /profile перенаправляет на страницу входа;
- test_anonymous_statistics_forbidden — неаутентифицированный запрос к /statistics перенаправляется;
- test_employee_cannot_access_statistics — сотрудник без is_staff получает 302 или 403 при обращении к /statistics;
- test_supervisor_can_access_statistics — руководитель с is_staff = True получает HTTP 200 на /statistics;
- test_employee_cannot_access_categories — сотрудник не имеет доступа к /categories;
- test_supervisor_can_access_categories — руководитель получает HTTP 200 на /categories;
- test_employee_cannot_edit_other_task — сотрудник получает 302 или 403 при попытке редактировать чужую задачу;
- test_employee_can_edit_own_task — сотрудник получает HTTP 200 при обращении к редактированию собственной задачи.

Результаты запуска фаззинг-тестов представлены на рисунке 9.

Листинг 9 – Результаты запуска фаззинг-тестов

[ПЛЕЙСХОЛДЕР]

Результаты запуска тестов (python manage.py test Tasks)

Ожидаемый вывод: Ran N tests in X.XXXs – OK

Сохранить как: images/fuzz_tests_output.png

5.2 Размещение кода в системе контроля версий

5.2.1 Структура репозитория

Исходный код проекта размещён в репозитории GitHub по адресу:
<https://github.com/Arsenoks4132/TaskMetrics>

Репозиторий содержит полную структуру проекта, описанную в файле `README.md`, включая схему директорий, архитектурную схему, таблицу стека технологий, инструкцию по запуску через Docker Compose, описание переменных окружения и сведения о демонстрационных пользователях.

Файл `README.md` содержит следующие разделы:

- описание функциональности приложения по ролям;
- полное дерево директорий с описанием ключевых файлов;
- архитектурная схема прохождения запроса (от браузера до СУБД);
- таблица технологического стека;
- инструкция по запуску (`docker compose up -d --build`);
- список необходимых переменных окружения (`.env`);
- логины и пароли демонстрационных пользователей.

5.2.2 История коммитов

Для репозитория ведётся содержательная история изменений. Каждый коммит отражает логически завершённое изменение с описанием цели правки. История коммитов репозитория представлена на рисунке 10.

Листинг 10 – История коммитов репозитория GitHub



5.3 Контейнеризация приложения

5.3.1 Dockerfile

Сборка образа Django-приложения описана в файле `docker/django/Dockerfile`. Образ основан на `python:3.12`, устанавливает зависимости из `requirements.txt`, копирует конфигурационный файл `.env` и скрипт запуска `entrypoint.sh`. Скрипт запуска последовательно выполняет применение миграций базы данных, загрузку начальной фикстуры (при первом запуске) и старт Gunicorn-сервера.

5.3.2 Docker Compose

Оркестрация всех сервисов приложения описана в файле `docker-compose.yaml`. Состав сервисов представлен в таблице 5.

Таблица 5 – Сервисы Docker Compose приложения TaskAndTimes

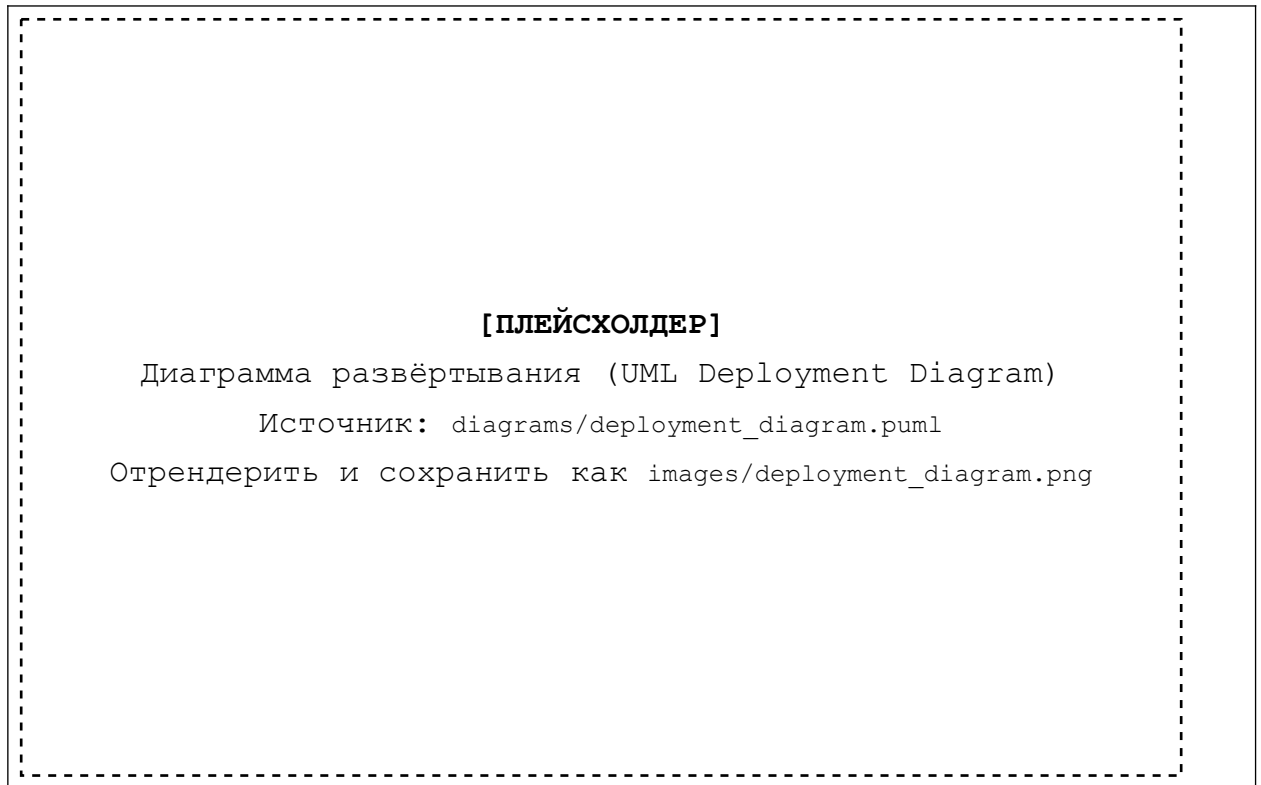
Сервис	Образ	Реплики	Роль
django_backend	python:3.12 (кастомный)	3	Сервер приложений (Gunicorn + Django)
postgres_db	postgres:17	1	Основная СУБД
nginx_frontend	nginx:1.27	3	Веб-сервер, отдача статики
traefik_balancer	traefik:v3.2	1	Балансировщик нагрузки, точка входа
redis_cache	redis:latest	1	Кэш страниц

Django-контейнеры и Nginx-контейнеры запускаются в трёх репликах для обеспечения горизонтального масштабирования. Traefik распределяет

входящие запросы между репликами по алгоритму round-robin. PostgreSQL и Redis запускаются в одном экземпляре.

Диаграмма развёртывания контейнеров на Yandex Cloud VM представлена на рисунке 11.

Листинг 11 – Диаграмма развёртывания приложения на Yandex Cloud



5.4 Развёртывание на Yandex Cloud

5.4.1 Процесс развёртывания

Приложение развёртывается на виртуальной машине Yandex Cloud Compute Cloud с операционной системой Ubuntu. Процесс развёртывания включает следующие шаги:

1) **Создание ВМ** — в консоли Yandex Cloud создаётся виртуальная машина с публичным IP-адресом. Открываются входящие порты 80 (HTTP) и 22 (SSH).

2) **Установка окружения** — на ВМ устанавливаются Docker и Docker Compose. Используются следующие команды: `sudo apt update && sudo apt install -y docker.io docker-compose-plugin` и `sudo usermod -aG docker $USER`.

3) **Клонирование репозитория** — исходный код загружается с GitHub командой `git clone https://github.com/Arsenoks4132/TaskMetrics.git`.

4) **Конфигурация окружения** — создаётся файл `.env` с необходимыми переменными: секретный ключ Django, данные для подключения к PostgreSQL, IP-адрес сервера.

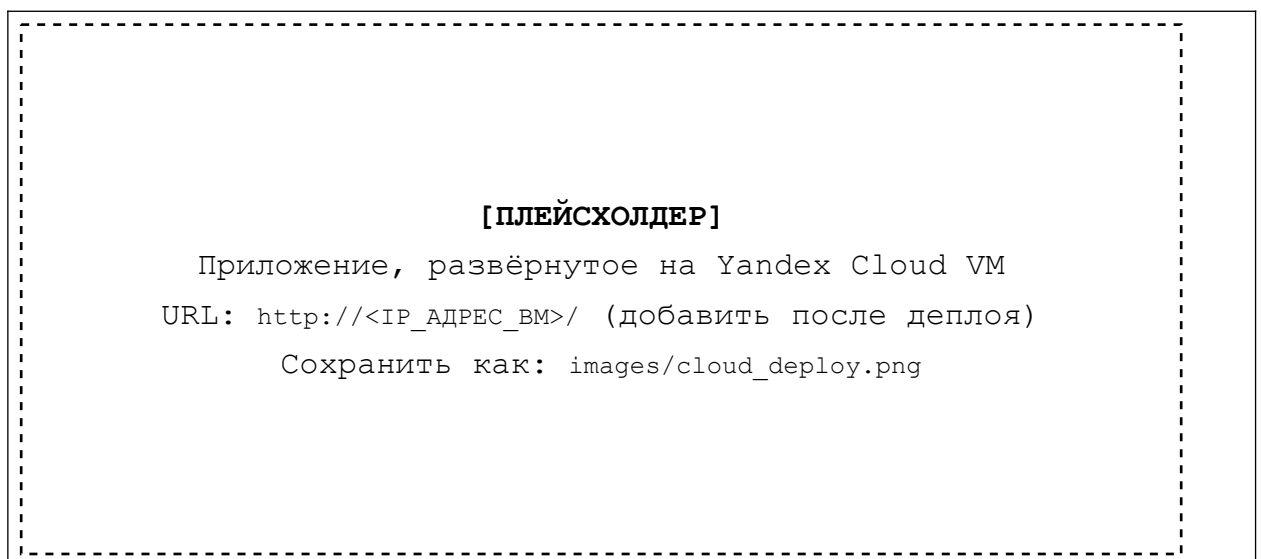
5) **Запуск приложения** — выполняется сборка и запуск всех контейнеров командой `docker compose up -d --build`.

6) **Проверка работоспособности** — приложение становится доступным по публичному IP-адресу ВМ через браузер. При первом запуске автоматически выполняются миграции и загружается тестовая фикстура с демонстрационными данными.

5.4.2 Результат развёртывания

После завершения развёртывания приложение доступно по адресу публичного IP-адреса виртуальной машины Yandex Cloud. Работающее приложение в облаке представлено на рисунке 12.

Листинг 12 – Приложение TaskAndTimes, развёрнутое на Yandex Cloud



5.5 Выводы по главе

В ходе выполнения данной главы было реализовано фаззинг-тестирование приложения с использованием библиотеки Hypothesis. Написаны четыре класса тестов, охватывающих граничные значения числовых полей, произвольные строковые данные в текстовых полях, валидацию форм и проверку ролевой модели на попытки несанкционированного доступа.

Исходный код проекта размещён в репозитории GitHub <https://github.com/Arsenoks4132/TaskMetrics> с содержательной историей коммитов и

развёрнутым файлом `README.md`. Приложение полностью контейнеризировано с помощью Docker: образ сервера приложений описан в `Dockerfile`, оркестрация пяти сервисов — в `docker-compose.yaml`. Процесс развёртывания на виртуальной машине Yandex Cloud сведён к четырём командам и не требует ручной настройки каждого сервиса.

ЗАКЛЮЧЕНИЕ

В рамках курсовой работы по дисциплине «Проектирование и разработка клиент-серверных приложений» было спроектировано и разработано полнофункциональное клиент-серверное приложение управления рабочими процессами TaskAndTimes. Поставленная цель достигнута: создано приложение, охватывающее все уровни клиент-серверной системы — от пользовательского интерфейса в браузере до облачного развёртывания.

В ходе выполнения работы решены следующие задачи.

Проведён анализ предметной области управления рабочими процессами. Выделены основные участники системы, определены ключевые бизнес-процессы, проведено сравнение аналогов (Jira, Bitrix24, Clockify) и сформированы функциональные и нефункциональные требования к приложению.

Выбрана и спроектирована клиент-серверная архитектура на основе трёхуровневой модели: уровень представления (браузер), уровень логики приложения (Django, Gunicorn, Nginx, Traefik) и уровень данных (PostgreSQL, Redis). Архитектура описана с помощью UML-диаграмм: диаграммы вариантов использования, компонентной диаграммы, диаграмм последовательности аутентификации и добавления задачи, диаграммы развёртывания.

Определён и обоснован программный стек: Django 5.1 как серверный фреймворк с паттерном MVT, PostgreSQL как основная СУБД, Redis как кэш-хранилище, Nginx и Traefik как веб-сервер и балансировщик нагрузки, Docker с Docker Compose для контейнеризации. В качестве системы контроля версий выбран Git с хостингом на GitHub, для облачного развёртывания — виртуальная машина Yandex Cloud.

Разработаны клиентская и серверная части приложения. Реализован полный набор CRUD-операций: для задач сотрудников (Create, Read, Update, Delete) и для категорий задач (Create, Read, Update, Delete). Добавлено редактирование профиля пользователя и смена пароля. Реализованы аутентификация через сессионный механизм Django и ролевая модель с двумя ролями (сотрудник и руководитель). База данных заполнена

тестовыми данными через фикстуру. Валидация ролевой модели обеспечена на нескольких уровнях: миксины разграничения доступа, явные проверки `PermissionDenied` и валидаторы полей форм и моделей.

Проведено фаззинг-тестирование с использованием библиотеки `Hypothesis`. Написаны тесты для граничных значений числовых полей, произвольных строковых данных, валидации форм и попыток несанкционированного доступа.

Исходный код размещён в системе контроля версий GitHub по адресу <https://github.com/Arsenoks4132/TaskMetrics> с историей коммитов, файлом `Dockerfile`, конфигурацией `docker-compose.yaml` и подробным описанием структуры проекта в `README.md`. Приложение развёрнуто на виртуальной машине Yandex Cloud.

Практическая значимость результата состоит в том, что разработанное приложение представляет собой готовый к использованию инструмент управления рабочими процессами, пригодный для дальнейшего развития: добавления REST API, мобильного клиента, расширенной аналитики и системы уведомлений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Тузовский А.Ф. Проектирование и разработка web-приложений [Электронный ресурс]: учебное пособие для вузов ISBN978–5–534–515–8. Москва: Юрайт, 2021. 218 с. URL: <https://urait.ru/bcode/469982> (дата обращения: 12.05.2026).
2. Меле А. Django 4 в примерах [Электронный ресурс]: руководство ISBN978–5–93700–204–4. Москва: ДМК Пресс, 2023. 800 с. URL: <https://e.lanbook.com/book/348113> (дата обращения: 12.05.2026).
3. Персиваль Г., Грегори Б. Паттерны разработки на Python. TDD, DDD и событийно-ориентированная архитектура. Санкт-Петербург: Питер, 2022. 336 с.
4. Антонова И.И., Кашкин Е.В. Разработка web-сервисов с использованием HTML, CSS, PHP и MySQL [Электронный ресурс]: учебно-методическое пособие. Москва: РТУ МИРЭА, 2019. URL: <http://library.mirea.ru/secret/15052019/2022.iso> (дата обращения: 12.05.2026).
5. Jira Software [Электронный ресурс]. 2026. URL: <https://www.atlassian.com/software/jira> (дата обращения: 12.05.2026).
6. Битрикс24 [Электронный ресурс]. 2026. URL: <https://www.bitrix24.ru/> (дата обращения: 12.05.2026).
7. Clockify [Электронный ресурс]. 2026. URL: <https://clockify.me/> (дата обращения: 12.05.2026).
8. Tempo Timesheets [Электронный ресурс]. 2026. URL: <https://www.tempo.io/product/timesheets> (дата обращения: 12.05.2026).
9. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. Санкт-Петербург: Питер, 2021. 352 с.
10. Flask Documentation [Электронный ресурс]. 2026. URL: <https://flask.palletsprojects.com/> (дата обращения: 12.05.2026).
11. FastAPI Documentation [Электронный ресурс]. 2026. URL: <https://fastapi.tiangolo.com/> (дата обращения: 12.05.2026).
12. PipeOps. PostgreSQL and MySQL: Which is Better? A 2024 Comparison. 2024.
13. PostgreSQL Documentation [Электронный ресурс]. 2026. URL: <https://www.postgresql.org/docs/> (дата обращения: 12.05.2026).

14. Redis Documentation [Электронный ресурс]. 2026. URL: <https://redis.io/docs/latest/> (дата обращения: 12.05.2026).
15. Chihana S. Web Server Performance of Apache and Nginx: A Systematic Literature Review. 2018.
16. Johansson A. HTTP Load Balancing Performance Evaluation of HAProxy, NGINX, Traefik and Envoy with the Round-Robin Algorithm. 2022.
17. Годзурас Э. Docker Compose для разработчика [Электронный ресурс]: руководство ISBN978–5–93700–203–7. Москва: ДМК Пресс, 2023. 220 с. URL: <https://e.lanbook.com/book/348110> (дата обращения: 12.05.2026).
18. Docker Docs [Электронный ресурс]. 2026. URL: <https://docs.docker.com/> (дата обращения: 12.05.2026).
19. Django documentation [Электронный ресурс]. 2026. URL: <https://docs.djangoproject.com/en/5.1/> (дата обращения: 12.05.2026).
20. Хоффман Э. Безопасность веб-приложений. Санкт-Петербург: Питер, 2021. 336 с.